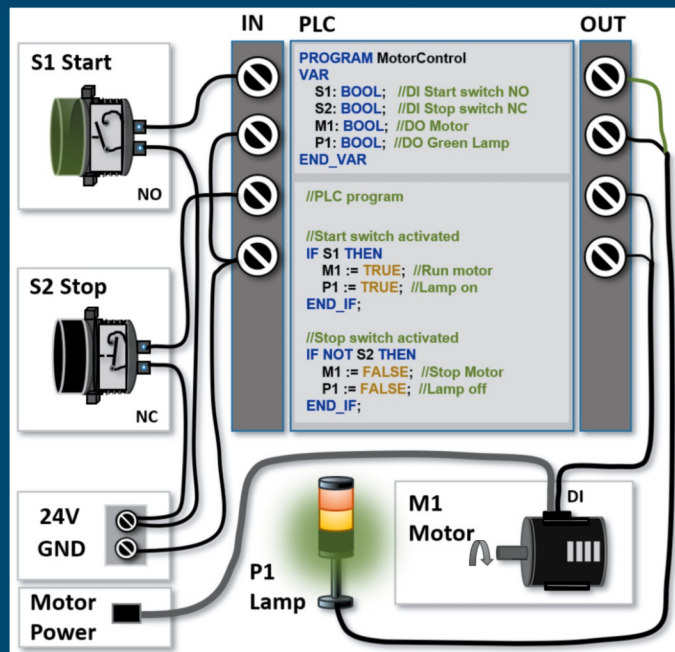


PLC Controls with Structured Text (ST)



IEC 61131-3 and best practice
ST programming

PLC Controls with Structured Text (ST)

PREFACE

When I started as Assistant Professor (lecturer) at Dania Academy, Randers, Denmark, one of my first tasks was to find books for the students at the Automation Technology degree course.

It turned out to be particularly difficult to find relevant material related to programming in Structured Text (ST). Books with hundreds of pages would only contain a few pages about ST programming, and only at a very theoretical level.

My students enjoy learning ST programming from practical examples and methods. In January 2017, I began work to fill this gap by developing new teaching material under the working title of:

"Get Started with Structured Text"

Whilst using the material in my lectures, it has continuously been updated and extended. There has been high demand for the material among my students, and I have now turned it into a book for other interested readers to benefit from.

It is my hope that you will enjoy this book.

I would like to extend my gratitude to my students, fellow lecturers and colleagues for feedback and inspiration.

Comments, complaints, compliments and suggestions are welcome and appreciated.

Please, send them to TomMejerAntonsen@gmail.com

First edition issued June 2018

This 4th edition has been updated with the bug fixes I received for both the English and Danish 3rd editions. Thank you for the many feedback and suggestions – without them, it would be difficult to improve the book. Additionally, there is now space for a few more examples and explanations.

Please enjoy!

Tom Mejer Antonsen

Randers, Denmark (March 2025)

Tom Mejer Antonsen

PLC Controls with Structured Text (ST)

IEC 61131-3 and best practice ST-programming

Die automatisierte Analyse des Werkes, um daraus Informationen insbesondere über Muster, Trends und Korrelationen gemäß § 44b UrhG („Text und Data Mining“) zu gewinnen, ist untersagt.

© 2018 - 2025 Tom Mejer Antonsen

4th Edition, March 2025

All rights reserved. No part of this publication may be reproduced, shared, or transmitted, in any form or by any means, including electronic or mechanical methods, photocopying, recording or otherwise, without the prior permission of the publisher. It is not permitted to use all or any part of this book to train AI models.

Illustrations and graphics: Tom Mejer Antonsen

Translated by: Katrine Bay Madsen

The original version (Danish): 1st edition, issued March 2018

*Publisher: BoD · [Books on Demand](https://www.bod.dk), Strandvejen 100, 2900 Hellerup, bod@bod.dk
Print: Libri Plureos GmbH, Friedensallee 273, 22763 Hamborg, Tyskland*

ISBN: 978-87-4306-968-3

Table of contents

1	INTRODUCTION.....	5
1.1	BACKGROUND FOR ST.....	6
1.2	PREREQUISITES FOR LEARNING ST PROGRAMMING	6
1.3	FOUNDATION OF KNOWLEDGE	7
1.4	ADVANTAGES OF ST PROGRAMMING	7
1.5	DISADVANTAGES OF ST PROGRAMMING	9
2	HOW THE PLC EXECUTES PLC CODE.....	9
3	COMMENTS IN THE PROGRAMMING CODE	12
4	DATA TYPES	14
4.1	ELEMENTARY DATA TYPES (INT, REAL, BOOL)	14
4.2	USER DEFINED DATA TYPES.....	18
4.3	ENUMERATED DATA TYPE, ENUM	18
4.4	STRUCTURED DATA TYPE, STRUCT	20
4.5	COLLECTION OF VALUES WITH SAME DATA TYPE, ARRAY	22
5	VARIABLE SCOPE.....	28
5.1	EXAMPLE: VARIABLES, SCOPE AND IO-MODULES.....	30
6	NAMING THE VARIABLES	31
6.1	VARIABLES WITH UNIT OF MEASUREMENT	36
6.2	VARIABLES WITH FIXED VALUES (CONSTANT)	38
7	OPERATORS, MATH AND LOGIC	39
7.1	ARITHMETIC OPERATORS (+, -, *, /).....	39
7.2	RELATIONAL OPERATORS (=, <, <=, >, >=, <>)	41
7.3	NUMERIC OPERATORS (MATH FUNCTIONS)	42
7.4	LOGIC OPERATORS (AND, OR, XOR, NOT).....	44
7.5	LOGIC, MATH FORMULAS AND USE OF PARENTHESES ()	45

8	VARIABLE ASSIGNMENT	46
8.1	MATH CALCULATIONS CHALLENGE	47
8.2	DIVISION BY ZERO	48
8.3	CALCULATING WITH REAL AND INT VARIABLES	49
8.4	DECIMAL ERRORS WHEN USING REAL	50
8.5	DATA COMMUNICATION (TRANSFER OF VARIABLES)	51
8.6	DATA TYPE CONVERSION FUNCTIONS	52
8.7	FINDING BINARY VALUES OF AN INTEGER (MASKING BIT)	53
8.8	VALVE MATRIX	54
8.9	ROUNDING A REAL TO 2 DECIMALS (2 DIGIT REAL)	55
9	BASIC ST PROGRAMMING	56
9.1	IF-THEN-ELSE STATEMENT	56
9.1.1	EXAMPLE: Motor control with self-holding relay	60
9.1.2	EXAMPLE: Manually operated tank control	62
9.1.3	EXAMPLE: IF-THEN-ELSE open and close valve	63
9.1.4	EXAMPLE: Robot control for packing items	64
9.2	CASE STATEMENT	66
9.2.1	EXAMPLE: CASE – Setting the motor speed	67
9.2.2	EXAMPLE: CASE – For executing programs	68
9.2.3	EXAMPLE: CASE – Recognizing numbers	69
9.3	ITERATION STATEMENT, LOOPS	70
9.4	FOR-DO STATEMENT	70
9.4.1	EXAMPLE: FOR – A loop running 4 times	72
9.4.2	EXAMPLE: FOR – LOOP and 3D ARRAY	73
9.4.3	EXAMPLE: Calculation of the average value	74
9.4.4	EXAMPLE: Find the lowest value in an array of numbers	76
9.4.5	EXAMPLE: Sorting numbers inside an ARRAY	77

10	SPLITTING UP THE PLC PROGRAM	78
10.1	PROGRAMMODULES	78
10.2	FUNCTIONS	80
10.3	FUNCTION (FC) AND FUNCTION BLOCK (FB)	82
10.4	DESIGN GUIDE FOR IMPLEMENTATION OF A FUNCTION	84
10.4.1	EXAMPLE: FC for conversion of temperature	87
10.4.2	EXAMPLE: FC to calculate average	88
10.4.3	EXAMPEL: FC for level measurement in tank	90
10.4.4	EXAMPEL: FC to linear scaling of sensor signal	92
11	WORKING WITH TEXT AND CHARS, STRING	94
11.1	EXAMPLE: FC WITH STRING	97
11.2	EXAMPLE: PROGRAM STRUCTURE FOR LANGUAGE CHANGE	98
11.3	STANDARD FUNCTIONS, STRING	100
11.4	EXAMPLE: FC FIND NUMBERS IN A STRING	103
11.5	FB: OPTIMIZE INSERTION OF VALUES INTO STRUCT	106
12	BUILT-IN STANDARD FUNCTIONS	108
12.1	FIRST PROGRAM EXECUTION: FIRST SCANBIT	108
12.2	EDGE DETECTION (ONE SHOT): R_TRIG, F_TRIG	109
12.2.1	EXAMPLE FB: One Shot rising detection	111
12.3	COUNTING FUNCTIONS: CTU, CTD, CTUD	112
12.3.1	EXAMPLE: Counting of items on a conveyor belt	114
12.3.2	EXAMPLE FB: Instrument pulse counter	115
12.4	REPEATED PROGRAM 'CALLS' AND TIMER DELAY: TON, TOF	116
12.4.1	EXAMPLE: Using the program scan as timer	118
12.4.2	EXAMPLE: Function block for Flashing Light	119
12.4.3	EXAMPLE FC: Time delay on digital alarms	120
12.4.4	EXAMPLE FC: Monitoring of analog values and alarms	122
12.4.5	EXAMPLE FB: Pulse pause function	124
12.4.6	EXAMPLE FB: A timer with a pause function	125

13	SPECIAL FUNCTIONS AND PROGRAM STRUCTURES.....	126
13.1	SIMPLE QUEUE STRUCTURE	126
13.2	FIFO – FIRST IN FIRST OUT.....	129
13.3	GENERATING RANDOM NUMBERS (RND, RANDOMIZE)	132
13.4	DIGITAL LOW-PASS FILTER (LP-FILTER)	134
13.5	SIMULATION SIGNALS FOR TESTING OF PROGRAM CODE	136
13.6	CONVEYOR BELT WITH SEQUENCE CONTROL.....	138
13.7	PUMP CONTROL WITH TWO PUMPS AND THREE FLOAT SWITCHES.....	142
13.8	PUMP CONTROL WITH SEQUENCE CONTROL.....	144
13.9	AUTOMATICALLY AND MANUALLY OPERATED PUMP CONTROL	146
13.10	CALCULATING TANK VOLUME, CYLINDER ON HEMISPHERE	147
13.11	PLC CONTROL FOR PUMPING WELL STATION WITH 6 PUMPS	150
13.12	EXAMPLE: HEATING OF LIQUID IN A TANK	158
13.13	EXAMPLE: FC TOGGLE SWITCH (TWO-WAY SWITCH).....	163
13.14	EXAMPLE: 3D CAR PARK CONTROLLED BY A ROBOT.....	164
13.15	EXAMPLE: CONFIGURABLE CAR WASH CONTROL	166
13.16	EXAMPLE: ADAPT PUMP SPEED TO SAVE ENERGY	170
13.17	PLC CONTROL OF ROBOT AND CNC MACHINE	175
14	FROM LADDER DIAGRAM TO ST-PROGRAMMING.....	178
15	BEST PRACTICE ST-PROGRAMMING	184
15.1	TABULATION OF TEXT AND PLACING OF SPACE	184
15.2	EMPTY LINES BETWEEN CODE.....	184
15.3	AVOID SPAGHETTI CODE	185
15.4	GOOD PROGRAM STRUCTURE	185
15.5	THE USE OF VARIABLES	186
15.6	MISCELLANEOUS.....	186
15.7	CODE SHARING ON THE INTERNET	187
15.8	OOP – OBJECT-ORIENTED PROGRAMMING	187
16	GUIDE AND HELP DURING ST-PROGRAMMING	188
16.1	GUIDE TO PROGRAMMING EXERCISES	188
16.2	PROGRAMMING AND TROUBLESHOOTING TIPS	191
16.3	MODULE TEST AND SIMULATION OF CONNECTED EQUIPMENT	194
17	INDEX	198

1 Introduction

This book provides an introduction to the programming language Structured Text (ST) which is used in **P**rogrammable **L**ogic **C**ontrollers (PLC) and **P**rogrammable **A**utomation **C**ontrollers (PAC).

The book can be used for all PLC types and PLC brands following the open international standard IEC 61131 part 3: programming languages.

In a Siemens PLC, the programming language is called **S**tructured **C**ontrol **L**anguage (SCL). SCL may differ slightly from programming in ST.

The book systematically describes basic programming, including advice and practical examples based on the author's extensive industrial experience.

Explanations to the PLC programming code with an emphasis on writing stable, robust, readable, structured and clean code are included in the book. The aim of the book is to enable the reader to write PLC code, which does not require a specific PLC type and can be reused across multiple types of PLCs.

It is recommended to read the entire book to gain an overview of its content, and then use the book as a reference moving forward.

The book was developed for the full-time "Academy Profession (AP) Graduate in Automation Engineering" course and the part-time "AP Degree in Automation and Operation" course at the Dania Academy, Randers, Denmark.

The book is based on the IEC 61131-3 standard. PLC suppliers and manufactures interpret the standard in different ways and not all follow the standard consistently. This means that some of the program examples in this book may not work properly in the PLC type you are using.

To get the most out of the book, it is recommended to try to program some of the examples covered in the book. Additionally, another book from the same author "*Collection of Exercises for PLC Programming*" is also recommended.

Unfortunately, the author is not available for support in connection with programming code within this book.

1.1 Background for ST

ST is a high-level programming language similar to Pascal Programming. Pascal Programming was widely used from 1980 to approx. 2000 – a period in which many companies started developing software for PC running on DOS (Disk Operating System), and later software running on Windows.

ST was developed and published by the International Electrotechnical Commission (IEC) in IEC 61131-3 International Standard in 1993. The standard consists of four PLC programming languages of which the Ladder Diagram (LD) language is the most well-known and commonly used. In addition to ST and LD, the other PLC programming languages include **F**unction **B**lock **D**iagram (FBD) and **S**equential **F**unction **C**hart (SFC).

Instruction List (IL) was previously part of the IEC 61131-3 standard.

Since about 2010 the usage of ST programming for PLCs has become widespread across Denmark, and many companies are now purely delivering PLCs programmed in ST. This means that the demand for ST programming capabilities across the industry has increased. This book is part of educating a workforce to fill this demand.

1.2 Prerequisites for learning ST programming

It is not necessary to know how to program in LD or the other PLC programming languages when learning ST programming. However, a certain level of knowledge of mathematics, mechanics, electronics, automation solutions and basic PLC is required to be able to learn ST programming.

Students with knowledge of a high-level programming language (e.g. VB, C++, C#, Python) will be able to learn ST relatively easy, due to the similarities in coding structures. The program execution inside a PLC is different compared to a traditional program or App running on a PC or a smartphone.

Like other text programming languages, the student can expect to be proficient in the language within three to five years.

1.3 Foundation of knowledge

The author has 30 years' industrial experience with the specification, development and delivery of complex control systems and supervision systems. Of the 25 years, the author has 7 years' experience with Pascal Programming and 12 years within automation solutions and systems involving PLC. This experience includes employment in four international companies and delivery of more than a thousand control system solutions in 20 different countries. This experience provides an important base for the content of this book.

In recent years the author has been teaching PLC Systems at degree level in Denmark. The students have from 0 to 20 years of practical and/or vocational experience within PLC, automation and technological services. The internet, the standard IEC/EN 61131-3 and a series of books on PLC programming have been utilized as inspiration for this book.

The material for this book was developed with feedback from lecturers and students attending the "Academy Profession (AP) Graduate in Automation Engineering" course and "AP Degree in Automation and Operation" course at the Dania Academy, Randers, Denmark. The content has been updated to answer the questions which the students typically ask during the course. The first edition of the book was published in March 2018. Since then, many students, teachers, businesses, and readers worldwide have shared suggestions, improvements and requests for later editions.

The author is senior lecturer/Associate Professor and has a Bachelor of Science in Electrical Engineering (B.Sc.E.E.) from Aarhus University School of Engineering (DK).

1.4 Advantages of ST programming

ST is a flexible and universal programming language. As ST programming code is based on text and not graphics like LD, the code can easily be copied between different PLC types, and even be sent by e-mail.

The ST programming code is similar to text sentences, and work is executed in the same way as a word processor program such as Microsoft Word which makes it easier to work on. Consequently, the same working methods are applied when using a word processor program or a text editor.

Because of its very structured nature, ST is ideal for tasks based on complex mathematics, code reuse or decision-making (e.g. automatic energy optimization, algorithms, data collection and regulation in process plants).

Having experience with ST programming will make learning other programming languages within PLCs and automation easier, including robotics, Visual Basic, or Python programming.

Within recent years, an increasing number of companies have switched to ST programming. This is due to the number of advantages offered by the ST programming language compared to the other three PLC programming languages (LD, SFC, and FBD).

The advantages of the ST programming language are:

- ST Programming code can relatively easily be copied between different PLC types and brands¹.
- It is the most convenient PLC language for mathematical calculations, formulas and algorithms², and for managing large amounts of data (Big Data).
- PLC solutions are more in demand today than 20 years ago³.
- Many widespread programming languages (C++, C#, VB, PASCAL) share similarities with the ST program structure.
- The other three PLC languages (LD, SFC, and FBD) require parts of the program to be written in ST anyway.
- Documentation of the ST PLC programming code requires less space during documentation, description and printing compared to other PLC programming languages.
- It is the easiest PLC language to version control via comments in the program code or via GIT or Subversion⁴.

The PLC programming language Instruction List (IL) has been used in many older PLC controls but is gradually being replaced with programs written in ST, as ST is the language closest to IL. Note that IL is named **STL** in a Siemens PLC.

Additionally, the author has found that many who are learning PLC programming for the first time prefer ST programming over LD programming, and those who have previously programmed for a PC or embedded computer prefer ST over LD.

¹) This is possible by using copy-paste and minor corrections. For example, a Siemens PLC uses the sign # before naming local variables, and Allen Bradley PLC uses a different syntax to make function 'calls'. The code can be sent by email.

²) Mathematical calculations are similar to mathematical formulas. Chapter 8.1 page 47.

³) There is more focus on energy optimization, automatic operation and data collection today. These are all solutions which requires more complex PLC coding than an ordinary 'relay/circuit breaker' with start/stop functions.

⁴) The tools GIT and Subversion are practical tools which allows the user to track (follow) corrections and extensions in the PLC Code. This makes it possible to commit current changes and fetch earlier versions of the PLC Code.

1.5 Disadvantages of ST programming

A big disadvantage is the fact that many technicians and electricians are only capable of programming in LD. It is difficult for them to understand ST program code because it is written in text and is not graphical like the LD program code¹⁾.

As a certain level of experience in structuring a program is required, programming in ST can easily be confusing.

Inexperienced people may have difficulties in fault-finding (debug) in an ST program.

Small (Micro) PLCs do normally not allow ST Programming.

It is not normally possible to apply ST Programming in a safety PLC²⁾.

ST programming is not something you learn just by watching YouTube videos, taking a short course, or looking at existing programs. While you can gain a basic understanding through videos and courses, the real learning happens when you write code, solve problems, and work on real projects. You should expect it to take between 3 and 5 years to become really good at ST programming.

2 How the PLC executes PLC code

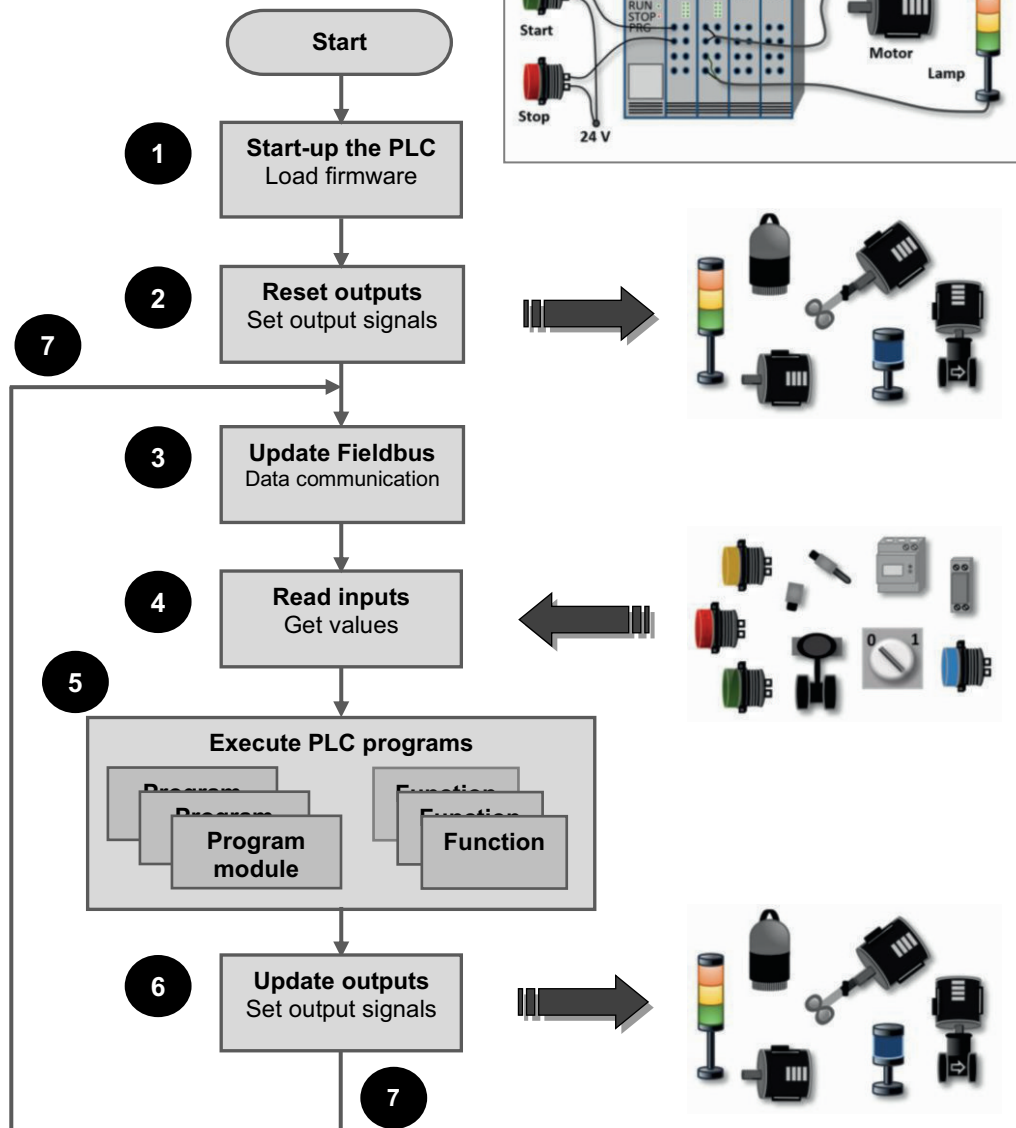
It is important to know how the PLC executes the program code when writing the program. A PLC executes the program in real-time, which means that the program modules must be executed within a short time-frame. The program modules are executed at a fixed time interval (the PLC scan time) e.g. 50 [ms]. Some of the fastest PLCs may have a scan time of 1 [µs].

Program modules can have different scan times e.g. 500 [ms] or 1000 [ms]. Some sensor input values do not change quickly (e.g. a temperature sensor). Which means that it is not necessary to have fast scan time for all program modules. A large program with many calculations takes a longer time to execute. This means that different programs will require different lengths of scan time. See also chapter 10.1, page 78.

¹⁾ To help learners who are already proficient (good) in the LD programming language, and would like to start programming in ST, Chapter 14 page 178) will be a good starting point. The chapter provides examples of different programs written in LD and the equivalent using the ST.

²⁾ A safety PLC is a separate PLC or a dedicated area in an ordinary PLC used to ensure the stop of the machine or plant when the emergency stop device is activated.

The basic mode of operation for a PLC:



The flow diagram shows the following points:

- 1 When power is turned on the PLC will start (boot up) and load the operating system, called firmware in a PLC system. This will ensure that the PLC program knows the connected hardware (HW).
- 2 After startup, all output modules are set to its initialization values. It is important that all outputs have the correct startup values so there are no unintended actions before the PLC program has started.
- 3 A data communication link has now been created via a network (fieldbus). Variables are received and sent out to other units (e.g. control panels, other control systems or instruments). There are many types of fieldbuses network systems. Some of the most used include Profibus, Profinet and Ethernet/IP. However, most fieldbuses are built with similar functionality, and they all work in similar ways.
- 4 Values from all sensors, switches, instruments and components on the machine or plant are now received from the connected input modules. A PLC automatically ensures that values are loaded and updated in memory, so they can be easily used in the PLC program. However, this must first be correctly configured in the PLC tool. How this is done is described in the user manual for the specific PLC type.
- 5 Dependent on the scan time all PLC programs will be executed once. Programs are split up as follows:
 - Program modules. See chapter 10.1, page 78
 - Functions. See chapter 10.2, page 80
 - Functions (FC) and Functions blocks (FB). Chapter 10.3, page 82
- 6 Programs must be split up in order to create a good program structure. Values are written to all output modules. Values could include new settings to motors/engines, valves, lamps and instruments. A PLC automatically retrieves values from its memory and sends them as signals to the physical outputs.
- 7 Step 3 to 6 will be repeated. This is one program scan. The execution of the program only stops if:
 - The PLC is set to STOP operation mode
 - If a run time error in the program occurs
 - The PLC is powered off or loses power unintentionally

3 Comments in the programming code

Comments are a very important part of programming. Comments in the programming code assist you and your colleague when later adding to the code.

Use comments to explain what a specific PLC code performs, so you can remember it later yourself. In many cases, the PLC code can be self-explanatory, therefore it is best only to make comments when the code is complex.

There are two types of comments in ST:

Line Comment:

```
// Line comment. Forward-slash is written in front of EVERY line.  
// Here you can write comments  
  
//You can write comments before a code section starts  
IF S1 THEN  
    K1:= TRUE; //Or write comments after the code at the same line  
END_IF;  
  
//Do not execute the code below  
//K2:= B2
```

A single-line comment always starts with // and is placed on the same line, either before or after the PLC code, as shown above. The PLC code here consists of the three lines that start with **IF** and end with **END_IF**.

Line comments can also be used to comment out PLC code so that it will not be executed (inactive). This is done by placing // in the beginning of the line. This can be an advantage rather than completely deleting the code. The code might still be needed later, or it could be beneficial to make some code inactive to troubleshoot an error elsewhere in the program.

Most PLC types use green text to display comments.

Line comments can only be placed on the same line in front of or after code.

Block Comment:

```
(* Block comment is initiated by a start parenthesis and a star. It is finalized by a star and end parentheses. They are used for making more lines of PLC code inactive i.e. create multiple comment lines *)
```

Comments placed between (* and *) are called block comments and are used to exclude multiple lines of code or to write comments filling up more lines.

Comments are essential at the start of any program module or function. This ensures that other programmers can understand the intended functionality of the program quickly and easily, without having to trawl through the entire code.

It is best practice to maintain a version log at the top of the program module which should include any changes made to code and the author of these:

```

////////////////////////////////////
// OP002 Parking house
////////////////////////////////////
// Action for each connected sensor
//
//*****
// Version 1.0, Created. Date 02.01.2025 TMA
// Version 1.1, TempVar3 changed 3.01.2025 TMA
// Version 1.2, Button S1 added 5.01.2025 TMA

IF S1 THEN //First line of PLC code
    K1:= TRUE;
END_IF;

//SetLamps(); //Do not run the SetLamps program module

```

A few PLC types cannot handle the special localized language characters such as æøå/ÆØÅ in the comment lines. As localized characters are not accepted by the PLC programming tool, it is therefore recommended to use the English alphabet in both the comment lines and the programming code. Due to this many companies choose to write their PLC code in English.

As seen above, the line comment is used to deactivate the **SetLamps()** code, which means that the PLC does not run the code. It can be an advantage to disable code when debugging. By excluding parts of the code, it becomes easier to find where the error is.

IMPORTANT! Do remember to correct the comments and version log if anything is later changed in the PLC code.

TIPS: Before starting the actual programming phase, you can use comments to gain a better understanding of the intended functionality of the program. This could help you to achieve more structure within the code, and help readers of the code understand it more easily.

4 Data types

Just like other programming languages, the IEC 61131-3 programming standard provides many different data types which include both elementary and complex ones. A data type defines how much memory capacity is needed by a variable value and by that, the largest and smallest value in the variable.

4.1 Elementary data types (INT, REAL, BOOL)

The following (examples) simple data types are standard in any PLC controller:

Data type	Bits	Numeral system	Note	Lowest and highest value	Example
BOOL (Bit)	1	Boolean (Binary)		FALSE/TRUE or 0 to 1	TRUE
BYTE	8	HEX (Hexadecimal)		16#0 to 16#FF	16#10
WORD UNIT	16	Binary		2#0 to 2#1111111111111111	2#0001000000000000
		HEX (Hexadecimal)		16#0 to 16#FFFF	16#1000
		BCD (Binary-Coded Decimal)		C#0 to C#999	C#998
		Unsigned Integer (positive numbers)		0 to 65535	564
DWORD (Double word)	32	Binary		2#0 to 2#11111111111111111111111111111111	2#10000001000110001011101101111111
		HEX (Hexadecimal)		16#00000000 to 16#FFFFFFFF	16#00A21234
		Unsigned Double word (integer)		0 to 4294967295 (4.29 billion)	435
INT (Integer)	16	Signed integer		-32768 to 32767	101
DINT (Double integer)	32	Signed double integer		-2147483648 to 2147483647 (2.1 billion)	107

Data type	Bits	Numeral system	Note	Lowest and highest value	Example
REAL (Floating-point number)	32	IEEE 754 Floating-point number (Decimal tal)	1	Lowest value: +/-3.402823E+38 Highest value: +/-1.175495E-38	1.234567e+13
LREAL (Long Real)	64	Dobbel Float (Decimal tal) IEEE 754		Lowest: -1.7976931348623E308 Highest: 1.7976931348623E308	3432.54
TIME (IEC time)	32	IEC time Step in 1 [ms] or	4	T#1ns to T#24d20h31m23s	TIME#10s T#10d14h11m23s T#5s12ms23us300ns
LTime	64	Step in 1 [ns]			
DATE (IEC date)	16	IEC day, step 1 day		D#1990-1-1 to D#2168-12-31	D#1996-3-15 DATE#1996-3-15
TIME_OF_DAY (Time)	32	Time in a step of 1 [ms]	4	TOD#0:0:0.0 to TOD#23:59:59.999	TOD#1:10:3.3 TIME_OF_DAY#1:10:3.3
CHAR WCHAR	8 16	ASCII characters (letter or sign)	2	'A', 'B' etc.	'E'
STRING		Text, letters	3	Up to 255 characters	"This is a text"

All variables must have a data type. If a variable is given a value outside the minimum and maximum value range of the data type, a run time error may occur and consequently the PLC may stop the program execution. This may again lead to strange behavior when executing the program (the program may seem unstable).

A few PLC types provide more data types than the ones listed above. In general, it is recommended use only a few data types so that the PLC code can be copied in an easier way to other PLC types. Some special data types such as **S7TIME**, **LWORD** and **ULINT** cannot be used by all PLC types. This means that copying PLC code with special datatypes, or upgrading to a larger PLC, may take a lot of work and risk introducing errors to the code.

The three most used data types are **BOOL**, **INT** and **REAL**. The reason why **INT** is used more often than **WORD** is that **INT** provides the same amount of data as the bit-size in a PLC making it a fast data type. On the other hand, if **REAL** is used, the PLC will auto-generate underlying machine code as the PLC can only work with integers.

The disadvantage of working with **INT** is when exchanging values between computers where e.g. one computer is a PLC running on a 16-bit operating system, and the second is running on a 64-bit operating system. The second computer could also be a small 8-bit computer (an embedded computer), used inside a sensor, a measuring instrument or equipment for analyzing process values. Read more in chapter 8.5, page 51.

Data type table notes

- 1) A **REAL** integer contains at most 7 influential digits. This means that if a variable is allocated the value of 1234.56789, the variable is not able to contain all digits. The value will consequently be changed to the value of 1234.567 (7 digits). Some PLC types use 8 digits: 1234.5678.
In some PLC types these data types are named **FLOAT**. Because computers may handle a **REAL/FLOAT** differently, some challenges can occur when communicating between several computers. In order to handle this, a **REAL** can be changed to an **INT** or **DINT** variable by multiplying by 100, and when data is received in another computer the variable has to be divided by 100. Using this method, a decimal number including 2 digits can be transferred without any problems. See more in Chapter 8.5, page 51.
- 2) ASCII characters are typically used when texts are needed to be written on e.g. user interfaces, data logging to files, communication between instruments, data from a keyboard or other PLCs. Since a PLC operates with integers only, each letter and symbol is represented by a number from the ASCII table.
The data type **CHAR** has 8 bits (may contain 255 different characters). A **CHAR** data type may typically be used for 1 to 5 different languages (countries). **WCHAR** has 16 bits and is applied for Unicode (ISO 10646, Universal Coded Character Set). Unicode is used for international PLC solutions.
WCHAR is typically used when the same PLC-code is applied in several countries with different languages in the user interface.
- 3) A **STRING** consists of an **ARRAY** of **CHAR** and is normally set to 255 characters (**CHARS**)
See above-mentioned note 2).
Furthermore, see chapter 11, page 94.
WSTRING is applied for Unicode (ISO 10646, Universal Coded Character Set) and consists of an **ARRAY** of **WCHAR**.
Note: Some PLC types provide a maximum of 80 characters in a **STRING**, if the **ARRAY** is not limited e.g. 10. It is good practice in programming to limit **ARRAY** so that unnecessary memory is not used.